BÀI TẬP CHƯƠNG 4

Bài tập 1. Tìm kiếm sách (Book Search)

Ở bài tập này, sinh viên cần hoàn tất tính năng **Book Search** đã thực hiện ở **Chương 1.** Sinh viên cần tạo một form tên là **SearchForm** để gửi và nhận chuỗi tìm kiếm từ biến **request.GET**. Form này có một hộp chọn để người dùng lựa chọn là tìm kiếm trong **tiêu đề (title)** hay **contributor**. Sau đó, tính năng này sẽ tìm tất cả các cuốn sách mà tiêu đề hoặc tên contributor (cả first_names và last_names) có chứa chuỗi cần tìm, và hiển thị danh sách kết quả sử dụng template tên là **search-results.html**. Người dùng không bắt buộc phải nhập chuỗi tìm kiếm, nhưng nếu có nhập thì chuỗi này phải dài *tối thiểu 3 ký tự*. (Lưu ý: do tính năng tìm kiếm sách có thể thực thi ngay cả khi thông điệp gửi đến bằng phương thức GET nên form sẽ luôn kiểm tra tính hợp lệ của dữ liệu ngay cả khi load lần đầu nếu ta đặt thuộc tính required cho hộp nhập.)

Sẽ có hai cách để thực hiện tìm kiếm. Cách thứ nhất là submit form tìm kiếm từ *các trang sử template chuẩn **base.html***, form này sẽ nằm ở *góc trên bên phải* của bất cứ trang web nào thuộc website. Khi đó, tính năng tìm kiếm chỉ thực hiện ***tìm trong tiêu đề cuốn sách (title).*** Cách thứ hai là submit form **SearchForm** từ trang sử dụng template **search-results.html** ở trên. Form này sẽ hiển thị **hộp chọn (ChoiceField)** để chọn tìm kiếm *trong tiêu đề hay contributor.*

Gợi ý các bước thực hiện như sau:

1. Tạo tập tin **forms.py**, import thư viện **forms** của Django rồi tạo lớp **SearchForm**.
2. **SearchForm** cần có các hộp nhập sau:
   - Hộp nhập văn bản **CharField** có tên là **search**, hộp này không bắt buộc nhập nhưng cần có ràng buộc *chuỗi nhập dài tối thiểu 3 ký tự*.
   - Hộp chọn **ChoiceField** có tên là **search_in** để chọn tìm kiếm trong tiêu đề (nhãn là *Title*) hay contributor (nhãn là *Contributor*). Hộp này là *bắt buộc chọn (required).*
3. Cập nhật view **book_search** để khởi tạo một đối tượng **SearchForm** sử dụng dữ liệu từ biến **request.GET**.
4. Thêm mã lệnh đề tìm kiếm với model **Book** sử dụng **title__icontains** (tìm kiếm có phân biệt chữ hoa/thường) khi tìm kiếm trên tiêu đề cuốn sách. Tính năng tìm kiếm này chỉ được thực thi nếu *dữ liệu gửi từ form là hợp lệ* và *có chứa chuỗi tìm kiếm*. Giá trị **search_in** cần phải truy xuất từ thuộc tính **cleaned_data** (lấy từ phương thức gửi GET) do giá trị này có thể không tồn tại (với hình thức tìm kiếm từ hộp tìm kiếm ở góc trên bên phải các trang web). Giá trị mặc định của biến **search_in** là ***title***.
5. Khi tìm kiếm từ **contributor**, cần sử dụng **first_names__icontains** và **last_names__icontains**, và duyệt qua tất cả contributor, sau đó truy xuất các cuốn sách tương ứng với mỗi contributor tìm được. Tính năng tìm kiếm này cũng chỉ thực thi khi dữ liệu gửi từ form là hợp lệ và có chứa chuỗi tìm kiếm. Có nhiều cách thức để kết hợp kết quả tìm kiếm từ **first_names** và **last_names**. Cách dễ nhất là thực hiện 2 truy vấn độc lập,

một cái tìm từ **first_names** và một tìm từ **last_names**, sau đó kết hợp 2 kết quả (có loại trừ trường hợp trùng lặp).

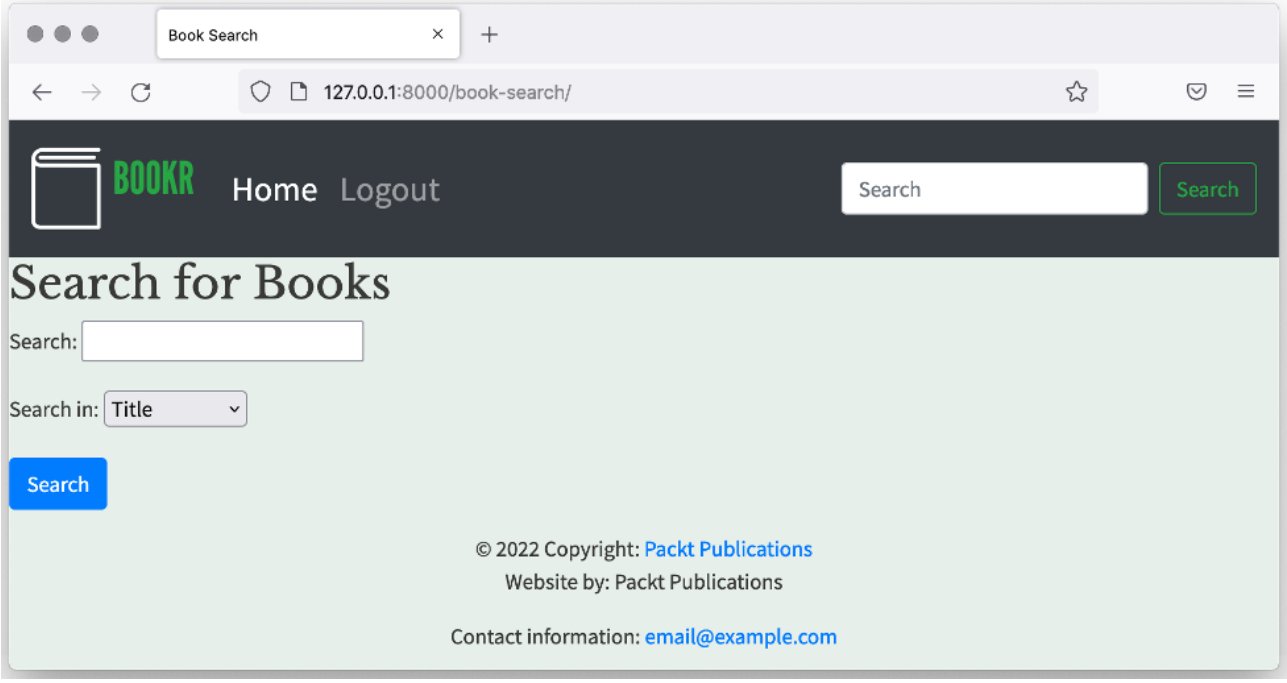
6. Cập nhật lại lời gọi hàm **render()** trong **view** để truyền thêm biến **form** và **các cuốn sách tìm được** qua biến ngữ cảnh (bên cạnh biến **search_text** đang có). Vị trí của tập tin template cũng đã có điều chỉnh ở Chương 3 nên ta cũng cần *cập nhật lại tham số thứ 2* của hàm render() này.
7. Cập nhật lại template **search_results.html** (đã tạo ở bài tập Chương 1) để kế thừa từ template **base.html**. Lưu ý xóa các nội dung dư thừa đi.
8. Thêm một tag **<title>** để hiển thị dòng **Search Results for <search text>** nếu dữ liệu từ form là hợp lệ, với **search_text** là chuỗi tìm kiếm. Trường hợp dữ liệu là không hợp lệ thì chỉ hiển thị cụm từ **Book Search**. Tag này cũng sẽ được đưa vào template **base.html** ở phần sau của bài tập này.
9. Thêm phần nội dung trang với tag **<h2>** ở đầu có nội dung là **Search for Books**, tiếp đến là thẻ **<form>** với các thiết đặt mặc định. Thêm một **nút submit** như đã làm trong các bài thực hành trước, nút này có thuộc tính **class** là **btn btn-primary**.
10. Phía dưới form, hiển thị một thông điệp là **Search Results for <search text>** nếu dữ liệu từ form là hợp lệ và chuỗi tìm kiếm **search_text** đã được nhập; ngược lại thì không hiển thị gì cả. Thông điệp này đặt trong thẻ **<h3>** và **<search_text>** nên đặt trong thẻ **.**
11. Duyệt và xuất tất cả các kết quả tìm kiếm. Với mỗi kết quả, hiển thị tiêu đề cuốn sách, **họ (last_names) và tên (first_names)** của các **contributor**. Tiêu đề sách liên kết đến trang **book_details**. Nếu danh sách kết quả tìm kiếm là trống thì hiển thị dòng **No results found**. Các kết quả cần đặt trong thẻ **<ul>** với thuộc tính **class** là **list_group**, mỗi kết quả đặt trong thẻ **<li>** với thuộc tính **class** là **list-group-item**. Trang này tương tự trang **book_list** nhưng hiển thị ít thông tin hơn (chỉ có tiêu đề và các contributor).
12. Cập nhật template **hase.html** để có thêm thuộc tính **action** trong thẻ **<form>** với giá trị lấy từ *tag template* **url**.
13. Đặt giá trị thuộc tính **name** của hộp nhập nội dung tìm kiếm là **search** và thuộc tính **value** là giá trị của biến **search_text**. Đồng thời đảm bảo ràng buuộc *chiều dài tối thiểu của văn bản nhập vào là 3*.
14. Trong tập tin **base.html,** thêm tag **<title>** như đã làm ở Bước 8. Thêm một *tag template* **block** trong thẻ **<title>** với nội dung là **Bookr**.

Sau khi hoàn tất bài tập này, khi mở trang **Book Search** theo URL http://127.0.0.1:8000/book-search/ sẽ có được trang web như **Hình I-1**.

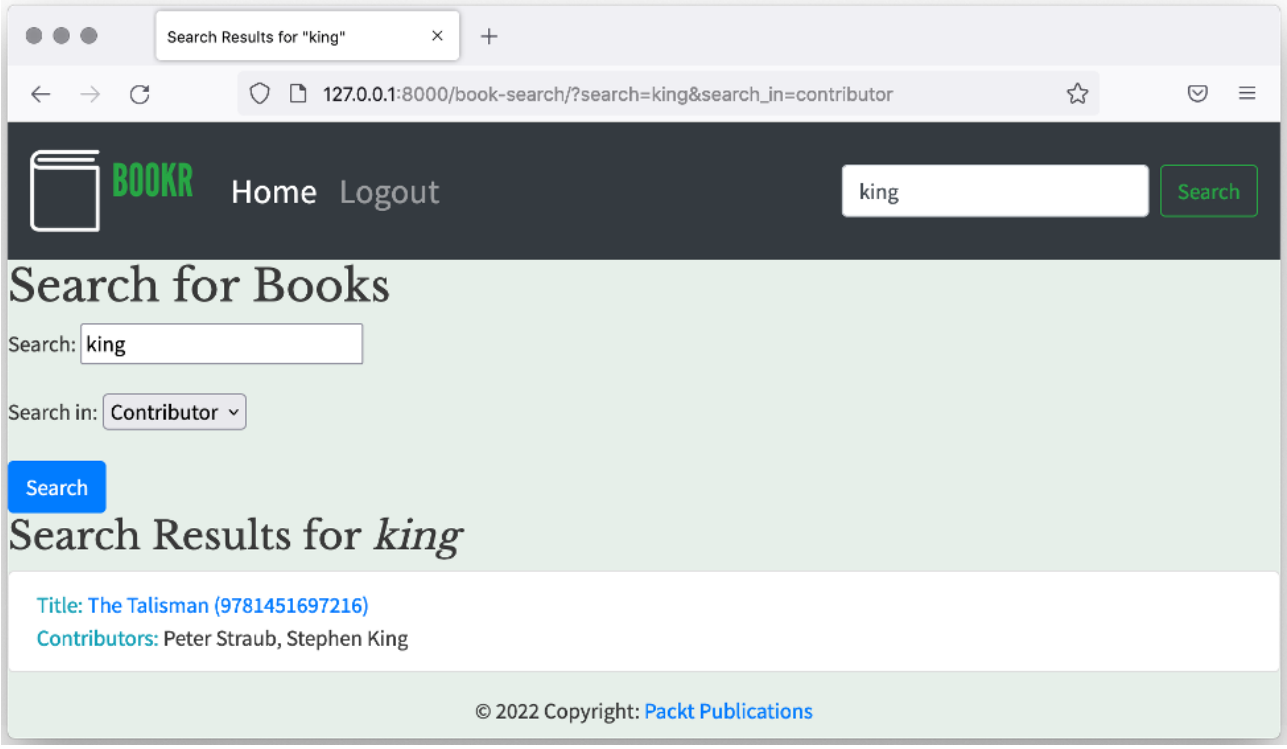
Khi nhập chuỗi tìm kiếm có chiều dài 2 ký tự vào ô tìm kiếm, trình duyệt sẽ không cho phép submit form.

Nếu không có kết quả nào khớp với chuỗi tìm kiếm thì trang web sé hiển thị thông điệp **No results found**.

Nếu tìm kiếm có ra kết quả thì giao diện trang tìm kiếm sẽ tương tự như **Hình I-2**.



Hình 1-1. Giao diện trang Book Search khi mở ra lần đầu



Hình 1-2. Trang Book Search khi tìm có kết quả

Bài tập 2. Tích hợp form cập nhật thông tin publisher

Trong bài thực hành VIII của chương này, ta đã thực hiện tạo form **PublisherForm** để tạo mới và chỉnh sửa thông tin của các **publisher**. Trong đó, ta đã sử dụng template riêng lẻ, không kế thừa từ bất cứ template nào, nên giao diện của trang web chứa form chưa tuân theo định dạng chung của website. Trong bài tập này, sinh viên cần xây dựng một trang thông tin chi tiết để hiện thị một form Django, tương tự như **form-example.html** nhưng *kế thừa từ*

tempate gốc của webstie. Trang template này sẽ nhận vào một biết để hiển thị loại model của dữ liệu cần chỉnh sửa. Sinh viên cũng cần cập nhật lại template gốc **base.html** để xuất được các thông điệp (message) của Django sử dụng thư viện **Bootstrap**.

Các bước gợi ý:

1. Đầu tiên, sinh viên cần từ chỉnh sửa tập tin **base.html**. Đặt khối **content** trong một thẻ ** ** để tạo bố cục tốt hơn với một số khoảng trống ở 4 cạnh. Có thể đặt **class** cho thẻ ** ** này là **"container-fluid".**
2. Xuất ra các thông điệp (message) trên template **base.html** bằng cách đặt một vòng lặp **for** (với thẻ **{% for %}**) vào sau phần mở thẻ ** ** (ở **Bước 1**) và trước khối **content**. Sinh viên nên sử dụng các lớp của thư viện **Bootstrap** để định dạng nội dung trình bày. Đoạn mã sau sẽ giúp xuất các thông điệp theo định dạng phù hợp với thư viện **Bootstrap**:

```
<div class="alert alert-{% if message.level_tag == 'error'
%} danger{% else %}{{ message.level_tag }}{% endif %}"
role="alert">
    {{ message }}
</div>
```

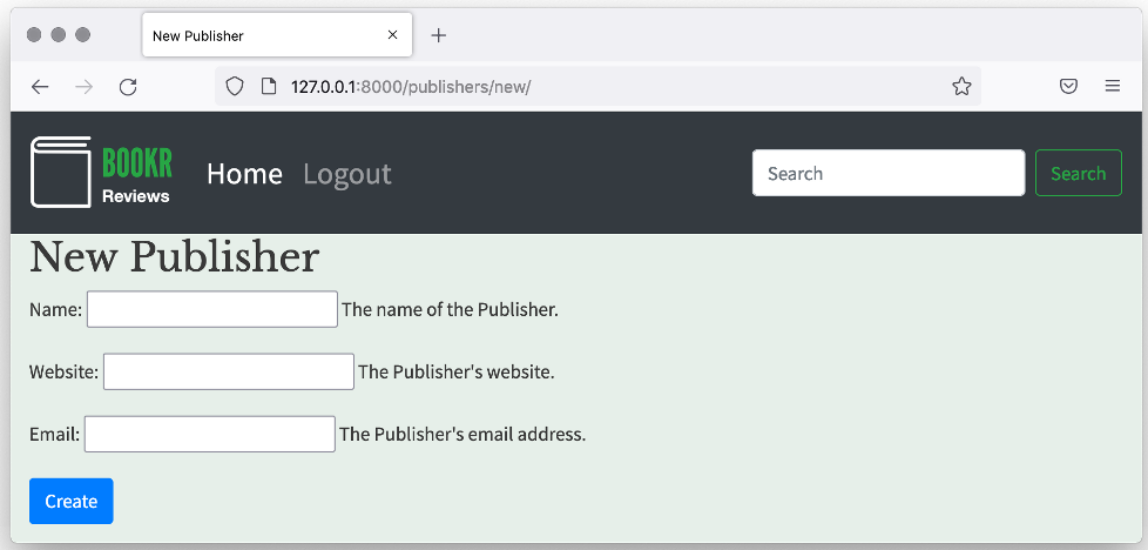
   Lớp **Bootstrap** và các thẻ **message** của Django có tên tương ứng trong hầu hết các trường hợp của loại thông điệp (thẻ **if** để xử lý các trường hợp này).

3. Tạo một template mới tên là **instance-form.html** trong thư mục **templates** của app **reviews**.
4. Template **instance-form.html** nên kế thừa từ template gốc **base.html**.
5. Biến ngữ cảnh truyền cho template này sẽ phải chứa một biến là **instance**. Nó sẽ mang giá trị là một đối tượng **Publisher** cần chỉnh sửa thông tin, hoặc **None** nếu ta cần tạo một **Publisher** mới. Biến ngữ cảnh cũng cần chứa một biến **model_type**, biến này chứa dữ liệu kiểu chuỗi để chỉ là **loại Model** (trong trường hợp này là **Publisher**). Sử dụng 2 biến này để xuất nội dụng cho thẻ **<title>** của template:
   - Nếu **instance** là **None** thì tiêu đề sẽ là **New Publisher**
   - Ngược lại, tiêu đề sẽ là **Editing Publisher <Tên publisher>**
6. Template **instance-form.html** cũng cần chứa một khối **tag** template **content** để ghi đè lên khối **content** của template gốc **base.html.**
7. Thêm một thẻ **<h2>** trong khối **content** này để xuất nội dung tương tự như nội dung của thẻ **<title>** ở **Bước 5**. Để có định dạng đẹp hơn, nên đặt **Tên publisher** vàotrong thẻ **.**
8. Thêm một thẻ **<form>** vào template với **method** là **post** và không cần đặt giá trị cho thuộc tính **action** (do dữ liệu của form sẽ gửi lại chính trang này).
9. Thêm một **token CSRF** trong form vừa thêm.
10. Xuất Django form (với tên biến ngữ cảnh nên là **form**) trong **form** trên sử dụng phương thức **as_p.**
11. Thêm một nút **submit** vào form. Nhãn của nút này tùy thuộc vào form hiện tại đang để nhập thông tin tạo mới hay để chỉnh sửa thông tin **Publisher**. Nếu tạo mới thì nhãn là

Create, còn chỉnh sửa thông tin thì nhãn là **Save**. Sinh viên có thể sử dụng các lớp của **Bootstrap** để định dạng nút này (có thể dùng **class** là "**btn btn-primary**").

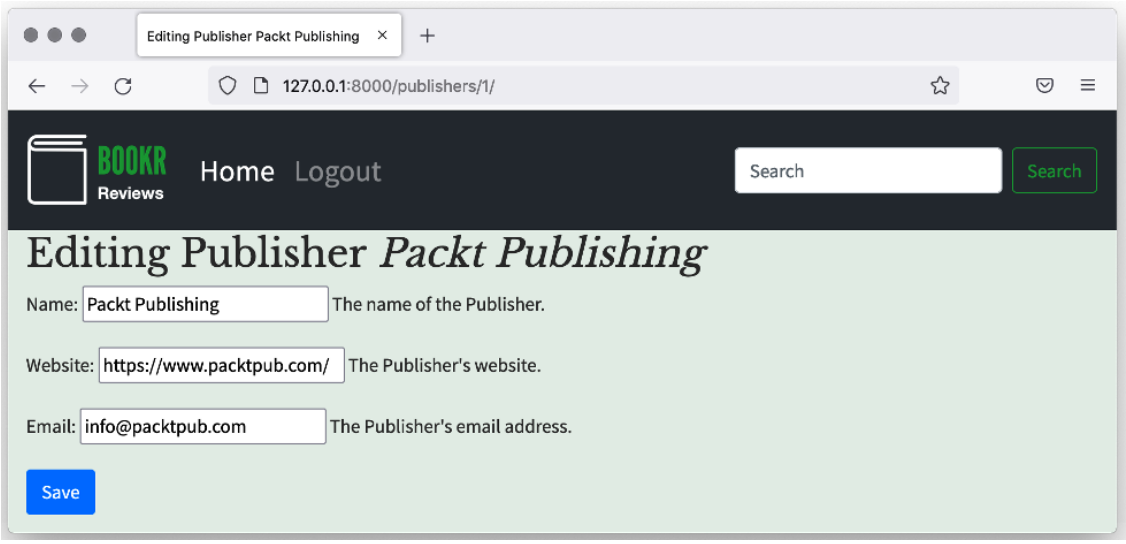
12. Trong tập tin **reviews/views.py**, không cần chỉnh sửa nhiều đối với hàm **publisher_edit**, chỉ cần điều chỉnh lại lời gọi hàm **render** để xuất template **instance-form.html** thay vì **form-example.html** và cập nhật lại biến **context** trong phần tham số. Biến **context** cần chứa thông tin đối tượng cần chỉnh sửa (**publisher**) và chuỗi **model_type** bên cạnh biến **form** đã có từ trước. Có thể bỏ biến **method**.
13. Template **form-example.html** không còn cần thiết nữa nên có thể xóa đi.

Giao diện của trang thêm mới publisher (URL là http://127.0.0.1:8000/publishers/new/) sẽ như **Hình 2-1**.



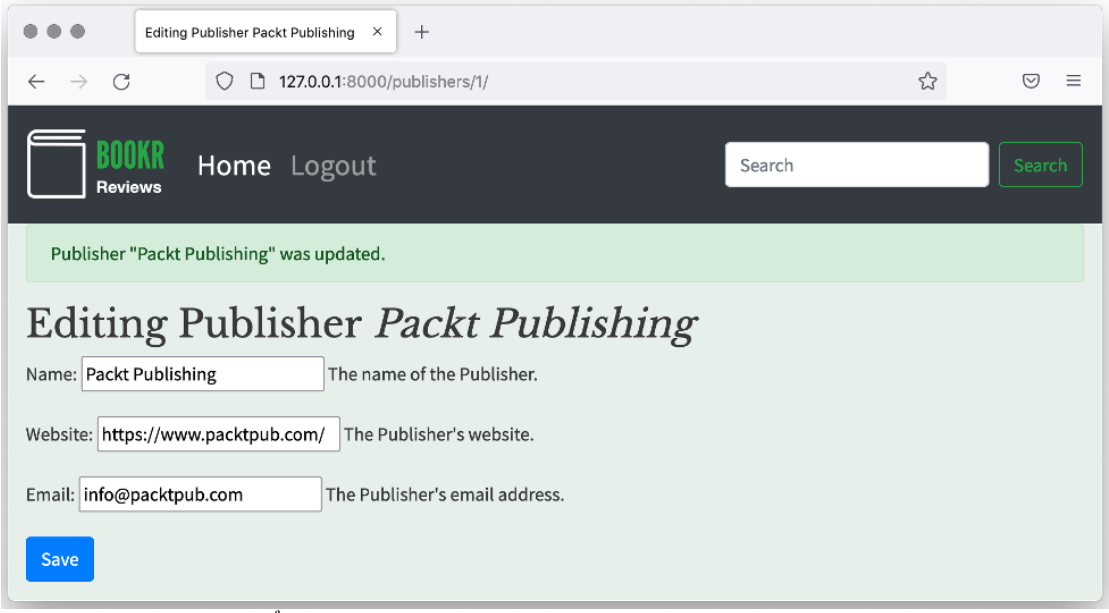
Hình 2-1. Trang thêm mới Publisher

Giao diện trang chỉnh sửa thông tin **publisher** (URL http://127.0.0.1:8000/publishers/1/) sẽ như **Hình 2-2.**



Hình 2-2. Trang chỉnh sửa thông tin Publisher

Sau khi lưu thông tin chỉnh sửa, giao diện sẽ như **Hình 2-3**.



Hình 2-3. Trang hiển thị thông điệp lưu thành công sau khi lưu thông tin chỉnh sửa

Bài tập 3. Giao diện trang tạo Review

Nội dung thực hiện để tạo form nhập thông tin **publisher** ở **Bài tập 2** là khá phức tạp, nhưng đây là cơ sở quan trọng để có thể dễ dàng tạo các form nhập thông tin cho các lớp model khác của website. Tên gọi template **instance-form.html** được dùng ở đây cũng để chỉ sự khái quát là ta có thể dùng template này trong nhiều view nhập thông tin khác nhau.

Trong bài tập này, sinh viên tạo một **ModelForm review** rồi thêm một hàm View **review_edit** để tạo mới hoặc chỉnh sửa thông tin của **Review**. Sinh viên có thể *tái sử dụng* template **instance-form.html** ở **Bài tập 2** và truyền các biến ngữ cảnh khác nhau để làm việc với Model **Review**. Khi làm việc với **Review**, sinh viên sẽ cần làm việc trong ngữ cảnh của một cuốn sách nên hàm View **review_edit** cần nhận vào tham số là giá trị **khóa chính của Book**. Sinh viên sẽ phải truy xuất dữ liệu cuốn sách và gán thông tin này vào cho **Review** cần tạo mới.

Các bước gợi ý:

1. Trong tập tin **forms.py**, thêm một lớp **ReviewForm** là lớp con của lớp **ModelForm** với **model** là **Review** (lưu ý **import model Review**).
   ReviewForm cần loại trừ các trường **date_edited** và **book** vì người dùng không cần thiết đặt các giá trị này từ form. CSDL chấp nhận bất cứ giá trị **rating** nào, nhưng ta cần phải nhập giá trị **rating** bằng trường **IntegerField** với ràng buộc giá trị **tối thiểu là 0 và tối đa là 5.**

   Tạo một hàm view mới gọi là **review_edit**. View này nhận vào 2 tham số (sau tham số **request**) là **book_pk** (bắt buộc có) và **review_pk** (tùy chọn, mặc định là **None**). Truy xuất dữ liệu của **Book** và **Review** sử dụng **shortcut get_object_or_404** (gọi theo từng biến). Khi truy xuất dữ liệu **Review**, cần đảm bảo là **Review** là dành cho cuốn sách **Book** này. Nếu giá trị **review_pk** là **None** thì giá trị **Review** sẽ là **None**.

2. Nếu **phương thức** của **request** là **POST** thì khởi tạo một **ReviewForm** sử dụng **request.POST** và đối tượng **Review**. Lưu ý **import ReviewForm**.
   Nếu dữ liệu nhập từ form là hợp lệ thì lưu form nhưng cần đổi giá trị tham số **commit** từ **save** thành **False**. Sau đó đặt thuộc tính **book** của đối tượng **Review** thành **ID của Book** đã truy xuất thông tin ở **Bước 1**.

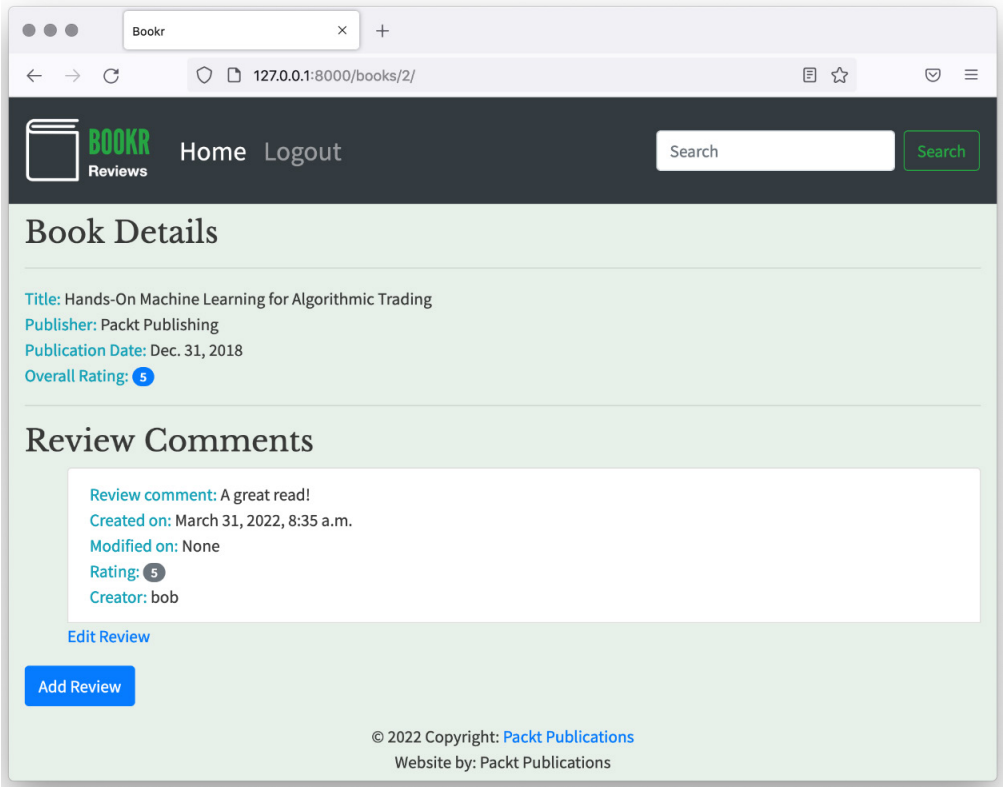
   Nếu đối tượng **Review** được cập nhật thay vì tạo mới, sinh viên cần đặt giá trị thuộc tính **date_edited** thành ngày giờ hiện tại sử dụng hàm **django.utils.timezone.now().** Sau đó lưu đối tượng **Review** vào CSDL.

3. Cuối cùng (trong phần xử khi dữ liệu là hợp lệ), đăng ký một thông điệp lưu thành công và chuyển trở lại trang **book_detail**. Do model **Review** không chứa trường thông tin phù hợp để hiển thị trong thông điệp này nên ta sử dụng **tiêu đề (title**) của cuốn sách. Nội dung thông điệp sẽ có dạng "**Review for <tiêu đề cuốn sách> created**".
4. Nếu phương thức của **request** không phải là **POST**, khởi tạo một **ReviewForm** và chỉ truyền đối tượng **ReviewForm**.
5. Xuất form ra template **instance-form.html**. Trong biến ngữ cảnh, cần có các biến như đã dùng trong chức năng **publisher_view** ở **Bài tập 2**, gồm: **form**, **instance** và **model_type** (**Review**). Và thêm 2 biến nữa là **related_model_type** (là **Book**) và **related_instance** (giá trị của đối tượng Book).
6. Chỉnh sửa form **instance-book.html**, thêm một vị trí để hiển thị thông tin của **related_instance**. Dưới thẻ **<h2>**, thêm một thẻ **<p>**, thẻ này chỉ hiển thị khi các biến **related_model_type** và **related_instance** có giá trị. Thẻ **<p>** hiển thị đoạn văn bản "**For <related_model_type> <related_instance>**". ***Ví dụ:*** *For Book Advanced Deep Learning with Keras*. Đặt phần **<related_instance>** trong thẻ **** để dễ đọc hơn.
7. Trong tập tin **urls.py** của app **reviews**, thêm một ánh xạ URL đến hàm view **review_edit**. Các ánh xạ /**books/** và **/books/<pk>/** đã có sẵn nên ta thêm:
   - /**books/<book_pk>/reviews/new/** cho **tạo mới** một review
   - **/books/<book_pk>/reviews/<review_pk>/** cho **chỉnh sửa** thông tin review

   Lưu ý đặt tên cho các ánh xạ này, ví dụ **review_create** và **review_edit**.

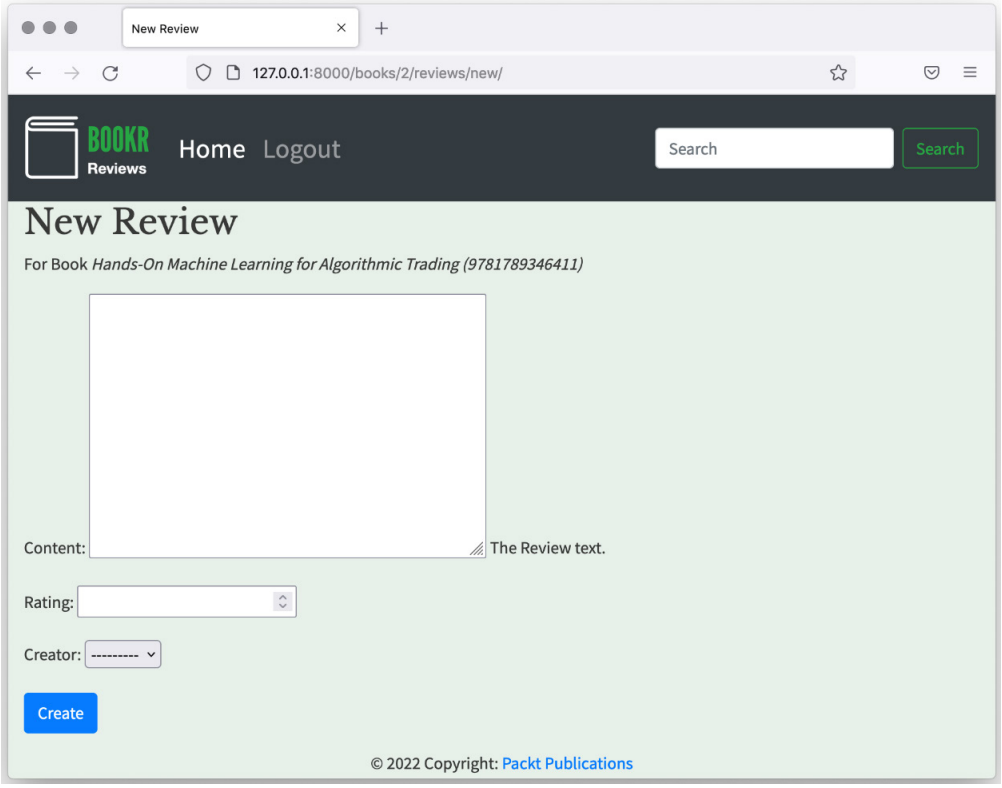
8. Trong template **book_detail.html**, thêm các liên kết (link) để người dùng có thể bấm chuột vào để tạo mới hoặc chỉnh sửa thông tin một review. Thêm 1 liên kết trong khối **content**, ngay trước thẻ đóng template **endblock**. Liên kết này sử dụng thẻ template **url** để liên kết đến tính năng **review_edit** khi ở chế độ tạo mới review. Đồng thời, sử dụng thuộc tính class với giá trị là "**btn btn-primary**" để hiển thị liên kết ở dạng một **nút bấm** (theo định dạng của **Bootstrap**). Nhãn của liên kết là **Add review**.
9. Cuối cùng, thêm một liên kết để chỉnh sửa thông tin một review, liên kết này cần đặt trong vòng lặp **for** để xuất hiện sau mỗi **review** của cuốn sách. Đặt liên kết này đặt sau tất cả các thẻ ** text-info,** liên kết đến tính năng **review_edit** với URL lấy từ thẻ template **url**. Ta cần cung cấp các tham số **book.pk** và **review.pk** của review cần chỉnh sửa thông tin. Nhãn của liên kết này là **Edit Review**.

Sau khi hoàn thành, trang **Review Comments** sẽ giống như trong **Hình 3-1.**



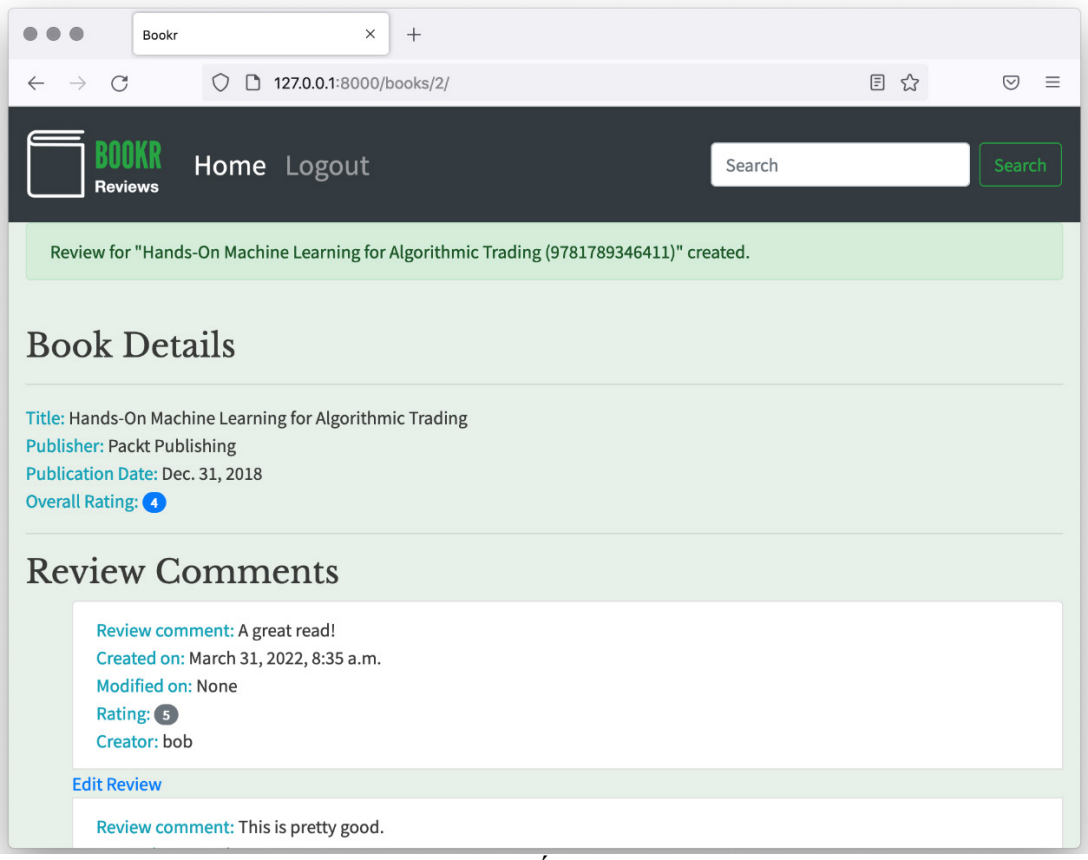
Hình 3-1. Trang Book Details với nút Add Review mới được thêm vào

Khi bấm chuột vào nút **Add Review** của trang **Book Details**, sẽ mở ra trang **Tạo mới Book Review** như trong **Hình 3-2.**



Hình 3-2. Trang Tạo mới Book Review

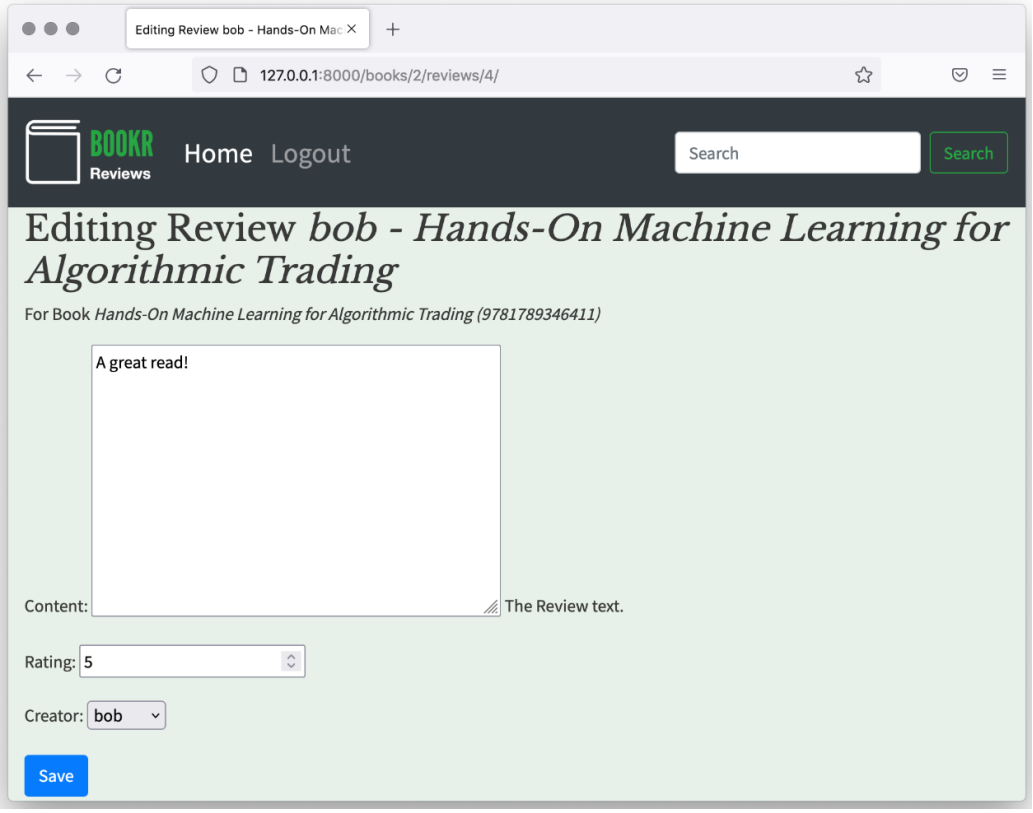
Nhập thông tin vào trang **Tạo mới Book Review** và nhấn vào nút **Create**, sẽ chuyển đến trang **Book Details** với thông điệp đã tạo mới thành công ở trên đầu trang như **Hình 3-3**.



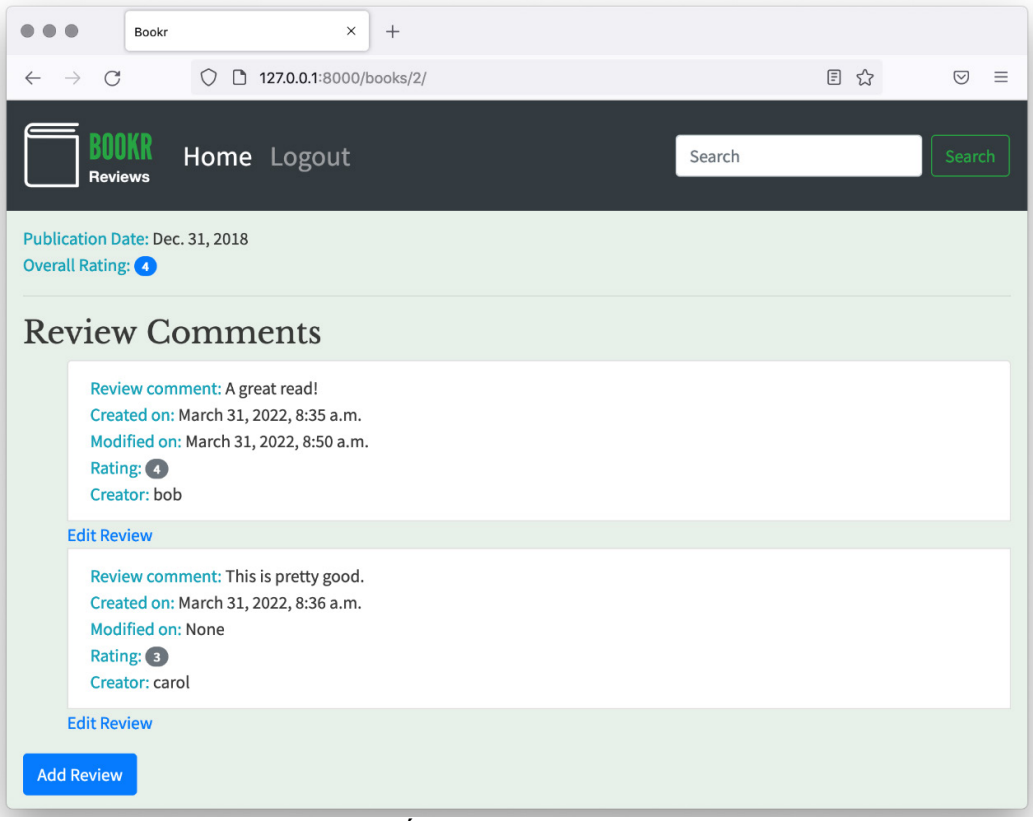
Hình 3-3. Trang Book Details xuất hiện sau khi tạo mới một Review

Ta cũng thấy link **Edit Review** để chỉnh sửa lại thông tin review nằm dưới mỗi review. Khi bấm chuột vào các link này sẽ mở ra trang chỉnh sửa thông tin review như **Hình 3-4**.

Sau khi lưu thông tin đã điều chỉnh của review, ta sẽ thấy giá trị ngày tháng **Modified on** của **review** đã được cập nhật trong trang **Book Details** như **Hình 3-5.**



Hình 3-4. Trang chỉnh sửa thông tin Review



Hình 3-5. Trang Book Details xuất hiện sau khi lưu thông tin chỉnh sửa Review xong